



Maestría en Inteligencia Artificial/ Maestría en Computación de Borde

Visión por Computadora III Trabajo Final

Alumnos:

Martín Brocca

Ariadna Garmendia

María Carina Roldán

Fecha: 5 de diciembre de 2025

Tabla de contenidos

Tabla de contenidos.....	2
Descripción del proyecto.....	3
Dataset utilizado.....	3
Análisis exploratorio (EDA).....	3
Enriquecimiento del dataset.....	4
Arquitectura general del sistema.....	5
Stack tecnológico.....	6
Implementación técnica.....	7
Preprocesamiento y enriquecimiento del dataset.....	7
Generación y almacenamiento de embeddings.....	7
Construcción del índice FAISS.....	8
Integración con MLflow y Databricks.....	8
Interfaz de usuario en Streamlit.....	9
Repositorio y estructura del código.....	10
Evaluación del sistema.....	10
Experimentos realizados.....	10
Métricas utilizadas.....	10
Visualizaciones generadas.....	11
Resultados cuantitativos.....	11
Pruebas con casos ambiguos y filtros dietéticos (CLIP vs. MiniLM).....	13
3. Fine-tuning de SigLIP.....	15
Resultados.....	20
Conclusiones y mejoras futuras.....	21
Futura incorporación del dataset Recipe1M+.....	22
Anexo. Planificación del equipo.....	23

Descripción del proyecto

El sistema consiste en un motor de búsqueda de recetas que acepta consultas mediante texto, ingredientes o imágenes. Además, provee filtros dietéticos inteligentes (recetas veganas, con y sin gluten, etc.) y capacidad para realizar sustituciones de ingredientes.

El sistema realiza búsquedas sobre 231K recetas en inglés usando embeddings de texto. La búsqueda se completa en milisegundos gracias a una implementación con FAISS.

Dataset utilizado

Para este trabajo se utilizó el [dataset Food.com](#), disponible en Kaggle, que contiene 231,637 recetas con información detallada sobre ingredientes, pasos de preparación, tags categóricos y metadatos nutricionales. El dataset original se encuentra en el archivo RAW_recipes.csv. Para las pruebas de funcionamiento finales del sistema, se utilizó un conjunto de imágenes recolectado en forma manual.

Se solicitó y obtuvo acceso oficial al dataset Recipe1M+ ($\approx$ 1 millón de pares imagen-receta etiquetados), pero su incorporación no fue posible dentro de los plazos y recursos disponibles (ver sección “Conclusiones y mejoras futuras”).

Análisis exploratorio (EDA)

Se realizó un análisis exploratorio del dataset con el objetivo de entender la estructura de los datos y la calidad del etiquetado original.

Receta 1: tomato and spinach garlic linguine with feta

Descripción: creamy, garlicky, easy to make, and just oh so yummy! you can use whatever type of linguine you prefer. i've made it with plain, spinach and chive, and garlic. when not using the garlic linguine, i ad...

Ingredientes: ['linguine', 'olive oil', 'frozen chopped spinach', 'broccoli florets', 'green onions', 'diced tomatoes', 'red pepper flakes', 'salt', 'pepper', 'dried oregano', 'feta']

Pasos: ['cook pasta according to package directions', 'in a large skillet , heat olive oil and add red pepper flakes and minced garlic , if using', 'when garlic is slightly browned , add green onions and bro...']

Tags: ['30-minutes-or-less', 'time-to-make', 'course', 'main-ingredient', 'preparation', 'healthy', 'main-dish', 'eggs-dairy', 'pasta', 'easy', 'beginner-cook', 'vegetarian', 'cheese', 'dietary', 'one-dish-...']

Figura 1. Ejemplo de receta del dataset food.com.

Durante este proceso se identificó que el etiquetado era inconsistente o impreciso: algunos tags categóricos no reflejaban correctamente el contenido de las recetas, o faltaban etiquetas básicas como por ejemplo gluten-free.

Enriquecimiento del dataset

Para mejorar la calidad de la información y la usabilidad del dataset, se implementó un sistema de etiquetado basado en los ingredientes, que identifica si las recetas contienen carne, pescado, lácteos, huevos, gluten, etc. Esto permitió generar tags inferidos más precisos.

A partir de RAW_recipes.csv se generó un dataset final que combina la información original con los nuevos tags y metadatos calculados, listo para ser usado en la aplicación de búsqueda.

El objetivo de este proceso de enriquecimiento de datos fue mejorar la calidad del motor de búsqueda y la interacción con la interfaz.



Figura 2. Etiquetas relacionadas con restricciones dietéticas (dataset original).



Figura 3. Etiquetas relacionadas con restricciones dietéticas (dataset enriquecido).

Arquitectura general del sistema

El sistema implementado es un motor de búsqueda multimodal de recetas que permite consultas tanto mediante texto libre como mediante imágenes del plato o de ingredientes. Se compone de los siguientes elementos:

- Interfaz web de usuario (Streamlit).
- Modelo multimodal preentrenado.
- Módulo retriever.
- Índice vectorial FAISS.
- Sistema de filtrado dietético y motor de sustituciones de ingredientes.

En la figura 4 se muestra el diagrama de bloques completo de la solución.

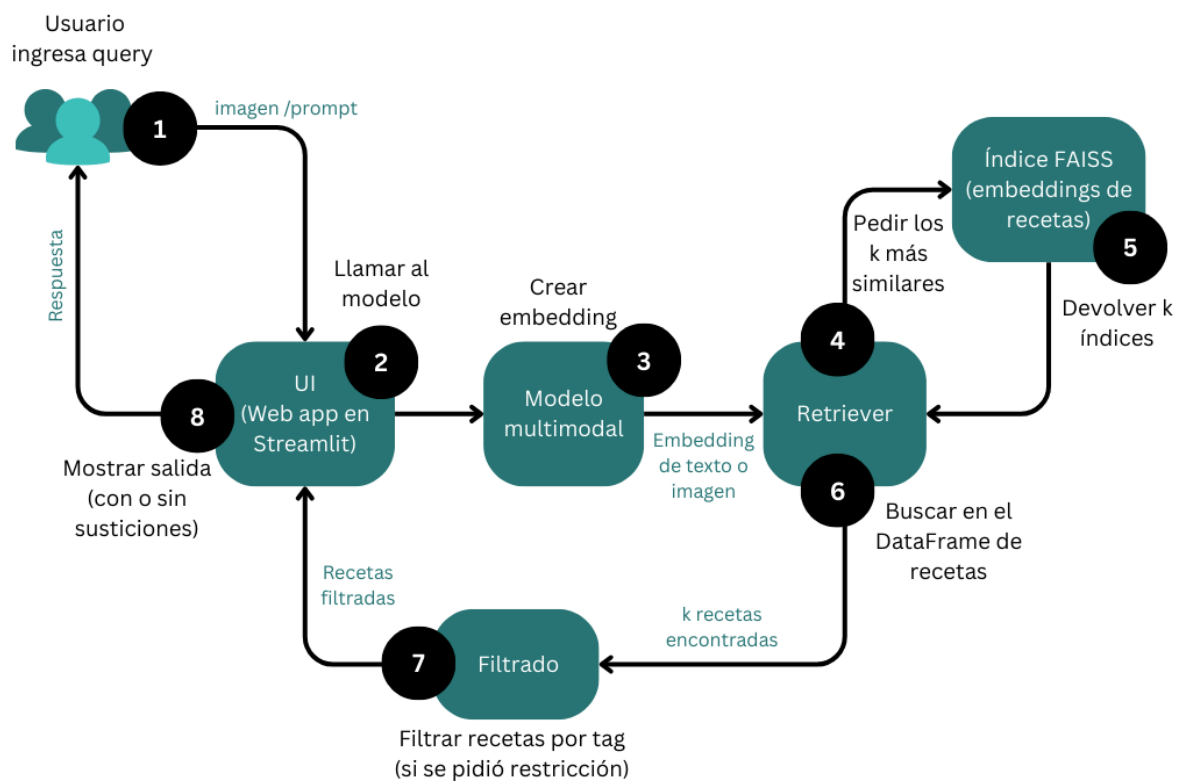


Figura 4. Arquitectura de la aplicación implementada.

El flujo de trabajo completo opera de la siguiente manera:

1. El usuario envía un pedido de receta en forma de prompt de texto o por medio de una imagen a través de la interfaz Streamlit.
2. La consulta se envía al modelo multimodal seleccionado.
3. El modelo genera un embedding.

4. El retriever ejecuta una búsqueda de similitudes en el índice FAISS.
5. FAISS devuelve los k índices más cercanos.
6. El retriever utiliza los índices para recuperar las recetas guardadas en el DataFrame principal.
7. El sistema aplica filtros dietéticos sobre los resultados candidatos.
8. La interfaz muestra los resultados finales junto con posibles sustituciones de ingredientes (si el usuario las solicitó).

Stack tecnológico

El proyecto utiliza las siguientes tecnologías y bibliotecas:

- Embeddings de texto: Sentence-Transformers con modelo all-MiniLM-L6-v2. Se trata de un modelo ligero y rápido especializado en búsqueda semántica de texto. Se usó como baseline para comparar contra CLIP. Genera vectores de 384 dimensiones en pocos milisegundos incluso en CPU.
- Embeddings multimodales: OpenAI CLIP (ViT-B/32 y ViT-L/14) + variantes de SigLIP. CLIP permite buscar recetas tanto con texto como con imágenes del plato o ingredientes usando el mismo espacio de embeddings.
- Búsqueda vectorial: FAISS (Facebook AI Similarity Search). FAISS es una biblioteca optimizada para búsquedas de similitud en millones de vectores.
- Interfaz web: Streamlit. Permite un prototipado rápido de la aplicación completa (subida de imágenes, filtros, resultados, sustituciones) solamente usando Python, por lo que se consideró ideal para este trabajo académico.
- MLOps y experimentación: MLflow integrado con Databricks. Esta tecnología permite el registro automático de todas las ejecuciones: métricas (Accuracy@k, latencia, etc.), parámetros, artefactos (gráficos PNG, índices FAISS) y comparaciones entre modelos. Facilita reproducibilidad y presentación de resultados.
- Procesamiento de datos: Pandas y NumPy. Estas bibliotecas fueron elegidas para la limpieza del dataset Food.com, para embeddings, enriquecimiento de tags dietéticos, cálculo de metadatos nutricionales y preparación del DataFrame final que se usa en producción.
- Deep learning framework: PyTorch (con soporte CUDA, MPS y CPU). Brinda flexibilidad total para cargar y ejecutar CLIP, SigLIP y Sentence-Transformers en GPU (NVIDIA), Apple Silicon (M1/M2) o CPU según el entorno disponible.

Implementación técnica

Preprocesamiento y enriquecimiento del dataset

Se partió del dataset público de Food.com. Los pasos realizados (figura 5) fueron:

- Corrección y completado de tags dietéticos ('vegan', 'vegetarian', 'gluten-free', 'low-carb', etc.).
- Normalización de nombres de ingredientes y títulos.
- Cálculo de metadatos nutricionales adicionales.
- Generación del DataFrame definitivo que se utiliza en tiempo de ejecución.

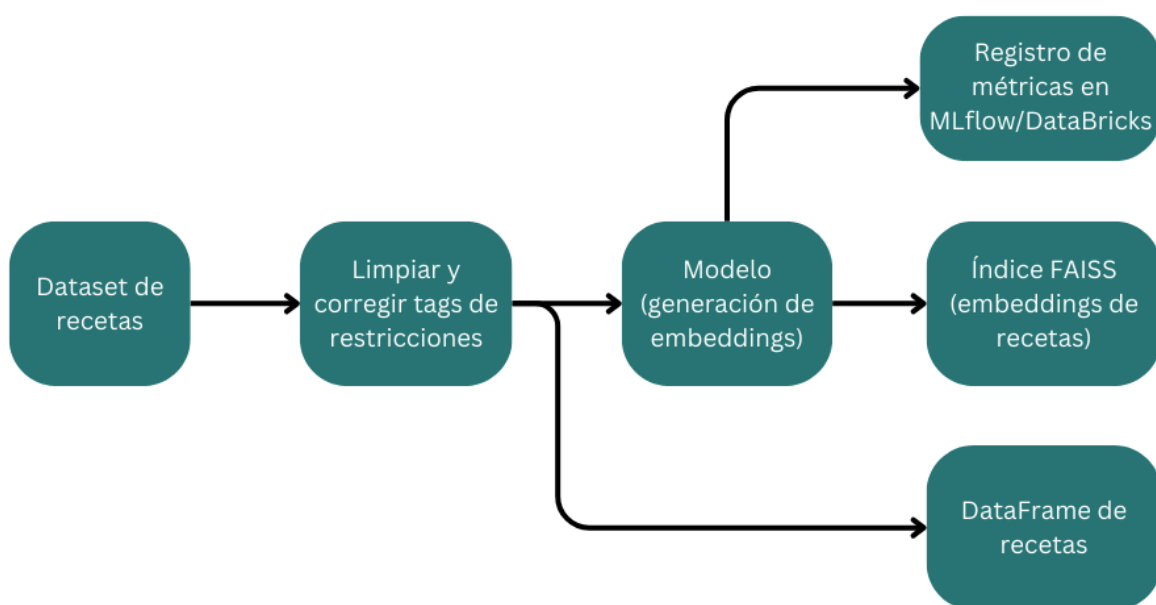


Figura 5. Pipeline completo de construcción del índice FAISS.

A continuación se describe cada bloque de la arquitectura en mayor detalle.

Generación y almacenamiento de embeddings

- Los embeddings de todas las recetas se generaron de forma offline utilizando PyTorch y el modelo seleccionado.
- Se registraron en MLflow las métricas de throughput, tiempo total y dimensionalidad.
- Los vectores resultantes se guardaron en formato .npy para su posterior uso.

Construcción del índice FAISS

- Se cargó el archivo .npy con los vectores.
- Se entrenó y construyó el índice FAISS.
- El índice se persiste en disco para su reutilización.

Integración con MLflow y Databricks

El sistema está integrado con MLflow Tracking (hosteado en Databricks), lo que garantiza trazabilidad y reproducibilidad de todos los experimentos. En cada ejecución se registran automáticamente los siguientes elementos:

- Estadísticas de preprocesamiento: número total de recetas procesadas, categorías dietéticas, etc.
- Métricas de embeddings: tiempo de generación, dimensionalidad, throughput.
- Resultados de evaluación: precision, recall, tasa de contaminación (palabras prohibidas), etc.
- Artefactos: gráficos comparativos en formato PNG.

Esta integración permite:

- Comparar de un vistazo todas las ejecuciones (CLIP vs MiniLM vs SigLIP).
- Reproducir exactamente cualquier resultado.
- Descargar gráficos y tablas listos para un informe.
- Identificar rápidamente la mejor configuración.

Para visualización en Databricks, se utiliza la siguiente url:

<https://dbc-02099e86-bcdd.cloud.databricks.com/ml/experiments/93934176252956>

Sin embargo, si se desea utilizar la instancia local, se puede ejecutar la interfaz de MLflow mediante el comando `mlflow ui`.

En Databricks (ver figura 6) los experimentos aparecen en la sección Machine Learning dentro de Experiments (Databricks → Workspace → Machine Learning → Experiments → [nombre del experimento]).

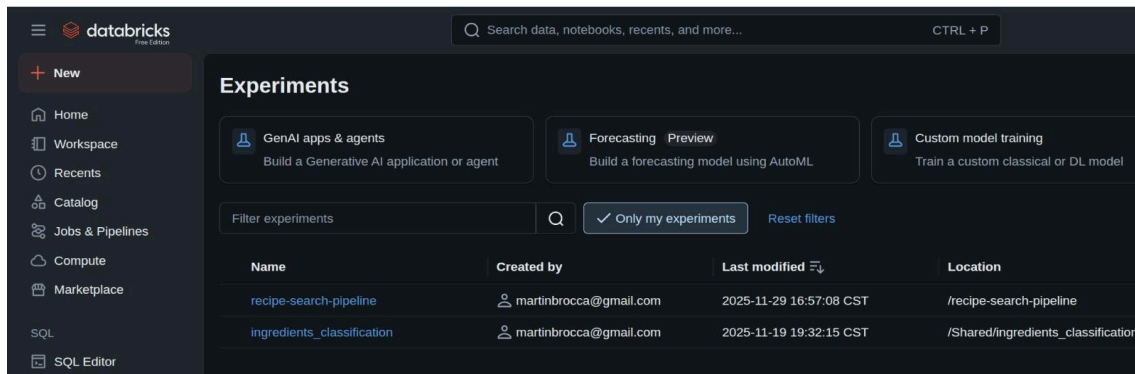


Figura 6. Documentación de experimentos en DataBricks.

Interfaz de usuario en Streamlit

La aplicación final (figura 7) incluye:

- Carga de imágenes o entrada de texto.
- Selección de filtros dietéticos.
- Opción de solicitar sustituciones de ingredientes.
- Visualización paginada de resultados con fotos, título, ingredientes y pasos.

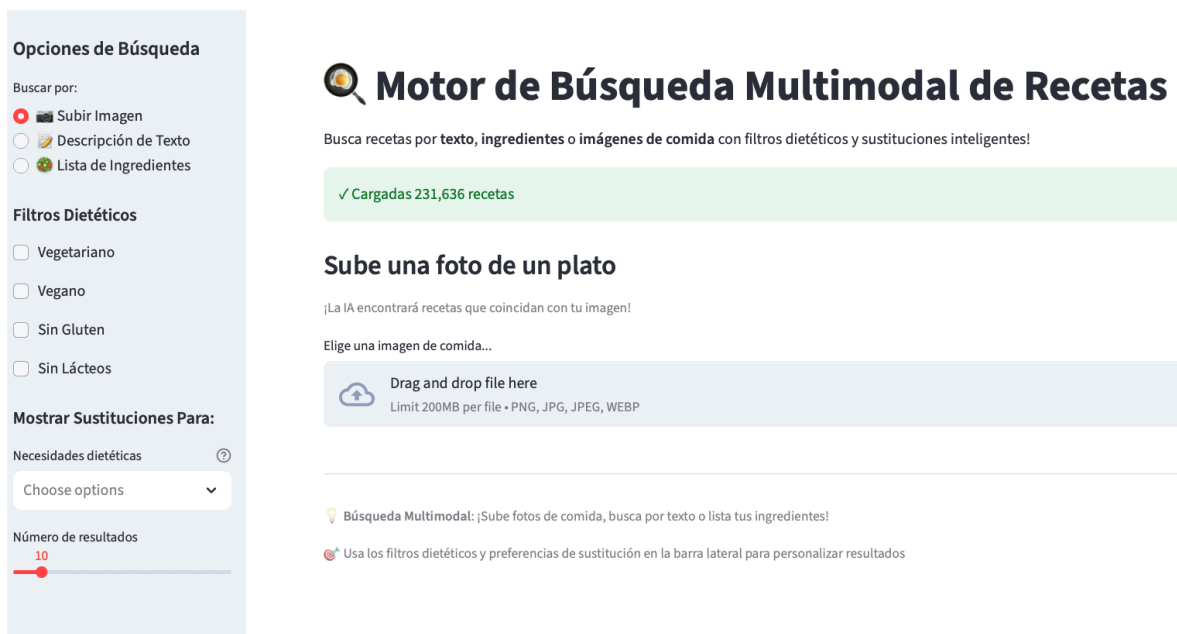


Figura 7. Interfaz de usuario del buscador de recetas.

Repositorio y estructura del código

Todo el código está versionado en un [repositorio](#) público de GitHub con una estructura de carpetas que siguen las buenas prácticas de desarrollo.

El sistema se encuentra preparado para despliegue en entornos de preproducción.

Evaluación del sistema

Para validar el rendimiento del sistema se diseñaron ensayos automáticos cuantitativos y evaluaciones cualitativas con usuarios reales. A continuación se describen los experimentos realizados y los resultados obtenidos.

Experimentos realizados

Se llevaron a cabo los siguientes tipos de evaluación:

1. Comparación multimodal vs. text-only. Se contrastó el mejor modelo multimodal (CLIP) contra un sentence transformer puro (all-MiniLM-L6-v2) utilizando únicamente consultas de texto.
2. Comparación entre modelos multimodales. Se evaluaron cuatro variantes de arquitectura CLIP/SigLIP:
 - CLIP-ViT-B/32 (512 dim) → modelo base
 - CLIP-ViT-L/14 (768 dim) → modelo grande
 - google/siglip-base-patch16-224
 - google/siglip2-base-patch16-224
3. Fine-tuning de SigLip (google/siglip-base-patch16-224) con un dataset pequeño, confeccionado especialmente para el trabajo.
4. Evaluación humana comparativa. Se implementó una interfaz side-by-side que permite comparar visualmente los resultados de dos modelos simultáneamente.

Métricas utilizadas

Para evaluar los resultados se eligieron las siguientes métricas:

- Text Accuracy@k: porcentaje de veces que la imagen correcta está en el top-k al buscar con texto. Mide calidad de ranking (text-to-image).
- Text Similarity@k: similitud promedio (coseno) entre la query de texto y las k imágenes recuperadas. Mide la calidad del espacio de embeddings.
- Image Similarity@k: similitud promedio (coseno) entre la query de imagen y los k textos (o imágenes) recuperados. Mide calidad del espacio de embeddings (image-to-text o image-to-image).

- Image Search Time: tiempo promedio de una búsqueda usando una imagen como query (incluye extracción de embedding + búsqueda). Mide velocidad real en producción.
- Text Search Time: tiempo promedio de una búsqueda usando texto como query (incluye extracción de embedding + búsqueda).
- Embedding Speed: velocidad de generación de embeddings (imágenes/segundo o ms por elemento). Es importante para indexado masivo y búsquedas en tiempo real.

Visualizaciones generadas

En cada ejecución, el sistema genera y registra automáticamente un conjunto completo de gráficos listos para usar en informes o presentaciones. Todos se guardan en formato PNG de alta resolución dentro del directorio experiments/ y se suben como artefactos a MLflow/Databricks.

Se generan los siguientes tipos de visualizaciones:

- Comparación general de modelos: incluye accuracy@k, matching de ingredientes, distribución de similitud, latencia y gráfico radar multidimensional. Estos gráficos permiten identificar fácilmente el modelo con el mejor balance general.
- Comparación específica de modelos multimodales (visión): muestra accuracy por variante de modelo, velocidad de generación de embedding y características de modelos. Ayudan a justificar la elección de un modelo.

Resultados cuantitativos

1. CLIP vs. MiniLM (consultas de texto)

La idea de este test fue evaluar qué tan bueno es el modelo multimodal para resolver consultas de texto, comparándolo contra un modelo especializado de tipo sentence-transformer. Para esto, el script `model_comparison.py` genera queries de texto de prueba y compara el desempeño obtenido al buscar recetas con embeddings creados tanto por el modelo más pequeño CLIP (genera embeddings de dimensión 512) como por el sentence-transformer MiniLM (crea embeddings de dimensión 384).

Los resultados obtenidos (figura 8) mostraron que CLIP es sorprendentemente bueno en consultas puramente de texto. Fue superior en todas las métricas, excepto en la relevancia de nombres de recetas, área en la que MiniLM proporciona mejor coincidencia. Esto tiene sentido ya que este se especializa en entender relaciones a nivel de palabra.

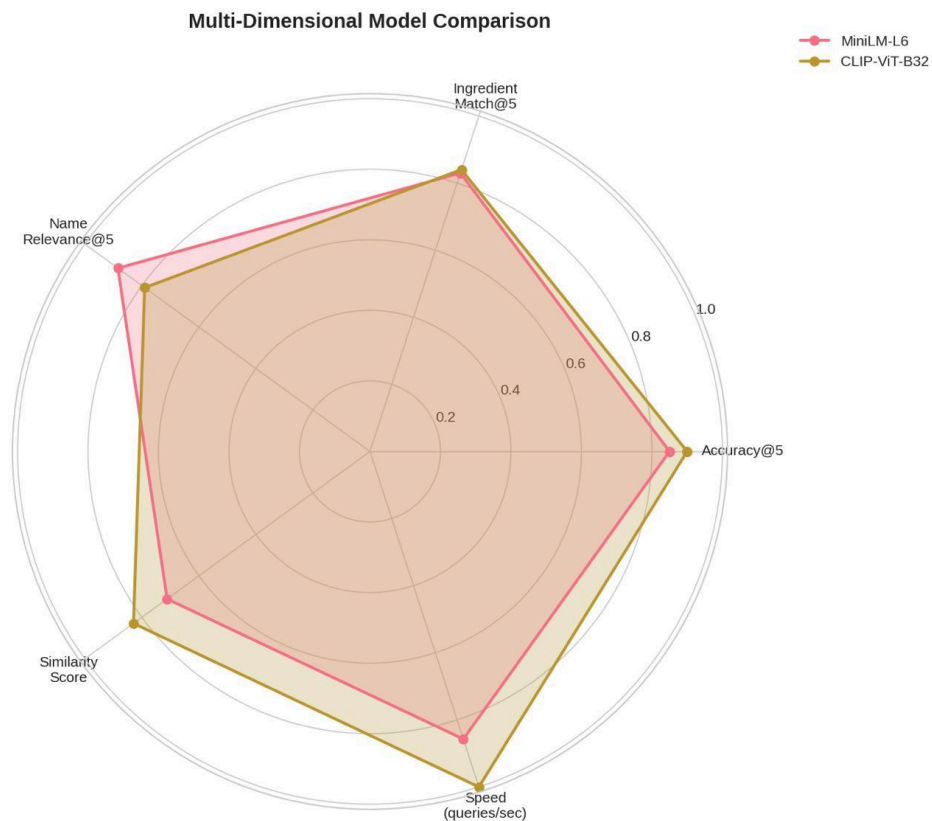


Figura 8. Diagrama de radar comparando los resultados.

Para mayor detalle, a continuación se muestran los valores obtenidos para todas las métricas evaluadas:

Métrica	MiniLM-L6	CLIP-ViT-B32
Accuracy@1	0,850	0,850
Accuracy@5	0,750	0,900
Accuracy@10	0,850	0,850
Ingredient Match@5	0,800	0,860
Name Relevance@5	0,883	0,743
Average Similarity@5	0,720	0,844
Latency (ms)*	244,7	82,1

(*) Obs: resultados obtenidos al ejecutar la prueba sobre una MacBook Pro M1, con 16 GB de memoria, 8-core GPU y aceleración con MPS.

De esta prueba se concluye que:

- CLIP supera claramente a MiniLM en la mayoría de métricas relevantes para búsqueda semántica real: mejor accuracy@5, mejor matching de ingredientes, scores de confianza más altos y menor latencia.
- MiniLM mantiene ventaja cuando la consulta coincide literalmente con palabras en los títulos de recetas.

Pruebas con casos ambiguos y filtros dietéticos (CLIP vs. MiniLM)

Se ejecutaron los siguientes casos de prueba representativos, que combinan ambigüedad semántica y aplicación de filtros dietéticos:

- Chocolate cake versus cookies: el sistema retorna correctamente pasteles.
- Carbonara versus otros platos italianos: discrimina correctamente el plato específico.
- Vegetarian burger: excluye correctamente ingredientes cárnicos.
- Gluten-free bread: falla ocasionalmente debido a tags dietéticos inconsistentes en el dataset.
- Vegan dessert: problema similar de inconsistencia en etiquetado.

El sistema alcanzó un 60% de tasa de éxito (3 de 5 tests) con los dos modelos, aunque CLIP mostró un desempeño levemente superior (valores de similitud coseno mayores y una velocidad más alta). Los fallos son atribuibles a limitaciones del dataset más que a limitaciones o deficiencias del modelo o la arquitectura. Los resultados de todas estas pruebas están registrados en MLflow.

2. Comparación entre variantes de modelos multimodales

Para comparar los 4 modelos multimodales, se desarrollaron pruebas automáticas con consultas controladas (archivo `vision_model_comparison.py`). Para este fin se definieron consultas de texto con ingredientes esperados:

Prompt/query	Esperado
"chocolate cake"	["chocolate", "cake"]
"pasta carbonara"	["pasta", "egg", "bacon"]
"chicken soup"	["chicken", "broth"]
"vegetable stir fry"	["vegetable", "soy"]
"banana bread"	["banana", "flour"]
"grilled salmon"	["salmon"]

"caesar salad"	["lettuce", "parmesan"]
"mushroom risotto"	["mushroom", "rice"]
"apple pie"	["apple", "cinnamon"]
"beef tacos"	["beef", "tortilla"]

Por otra parte, se realizaron pruebas con imágenes reales. Se utilizaron fotografías como las que se muestran en la figura 9.

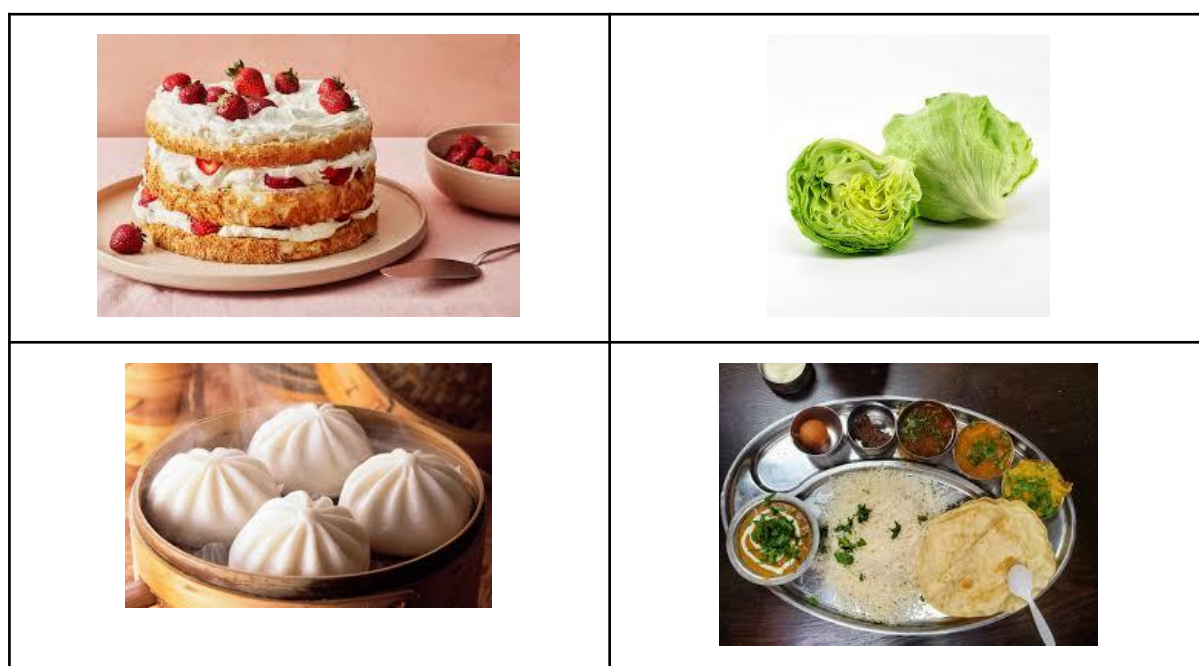


Figura 9. Ejemplos de imágenes de prueba.

Los resultados de las pruebas (ejecutados sobre MacBook Pro M1 al igual que en el caso anterior) se resumen en la siguiente tabla.

Métrica	CLIP-ViT-B/32	CLIP-ViT-L/14	SigLIP-Base	SigLIP V2
Dimensión embeddings	512	768	768	1152
Text Accuracy@5	0,900	1	1	0,9
Text Similarity@5	0,761	0,695	0,763	0,753
Embedding speed (recetas/seg)	338	193	255	56
Text search time [ms]	73,6	49	14,8	73,6

- Sorprendentemente, la versión más pequeña de CLIP (ViT-B/32) supera a la variante más grande (ViT-L/14) en este dominio específico, lo que sugiere que el modelo más pequeño generaliza mejor para recetas.
- Ambos sistemas mantienen búsqueda con latencia adecuada sobre 231K recetas, lo que resulta conveniente para aplicaciones interactivas.

3. Fine-tuning de SigLIP

Como tercer grupo de pruebas, se realizó un ejercicio de fine-tuning sobre el modelo `google/siglip-base-patch16-224` con un dataset pequeño construido en forma manual. Este se elaboró a partir del dataset [food-101](#) que ofrece 101k imágenes de comidas, pertenecientes a 101 categorías o clases distintas. Ejemplos de categorías del dataset food-101, son: `apple_pie`, `baby_back_ribs`, `baklava`, `beef_carpaccio`, `beef_tartare`, `pizza`, `hamburger`, `hot_dog`, `spaghetti_carbonara`, `sushi`, etc.

Con imágenes de food-101 y recetas de food-com, se confeccionó el dataset final con 21729 pares imagen-receta (localizado en la carpeta `data/processed/food101_recipe_pairs.json`).

A modo de ejemplo, se muestra la estructura de uno de los pares del archivo JSON:

```
{
  "image_path": "food-101/images/apple_pie/134679.jpg",
  "recipe": {
    "name": "Classic Apple Pie",
    "ingredients": ["apples", "sugar", "cinnamon", ...],
    "tags": ["dessert", "baked", "american"],
    "recipe_text": "Classic Apple Pie. Ingredients: apples, ..."
  },
  "category": "apple_pie",
  "quality_score": 0.85,
  "split": "train"
}
```

Los datos se repartieron de la siguiente manera:

Train	Validación	Test
17,383 pares (80%)	2,172 pares (10%)	2,174 pares (10%)

Además, se verifican las siguientes características:

Categorías: 86

Imágenes únicas: 8,600

Recetas únicas: ~18,000

Pares por categoría (promedio): 253

Distribución balanceada: Sí

Por otra parte, para el entrenamiento se utilizó la siguiente configuración:

- Épocas: 5
- Batch size: 32
- Learning rate: 5e-6
- Optimizador: AdamW

Durante el proceso se registraron:

- train_loss, eval_loss (cada 50 steps)
- learning_rate (cada step)
- epoch
- training_time

Además, se guardaron los siguientes artefactos:

- Pesos del modelo completo
- training_summary.json
- training_curves.png (pérdida y LR)

En la figura 10 se muestra un gráfico que compara el desempeño con el dataset de test, para distintos top-k.

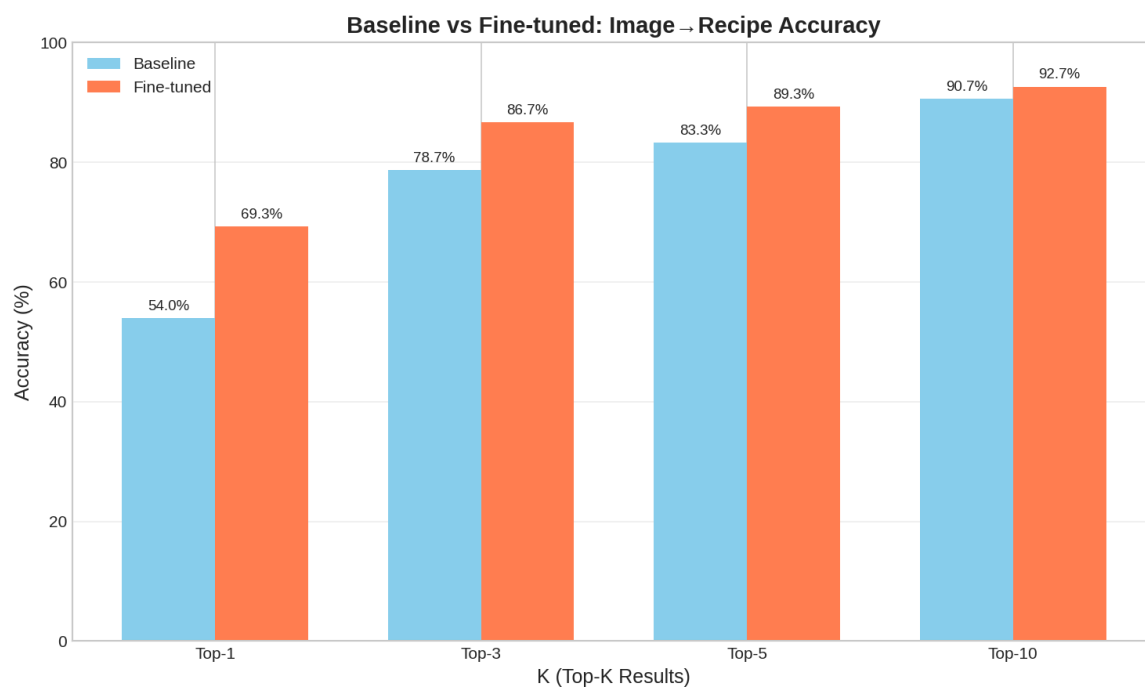


Figura 10. Comparación de desempeño de SigLIP antes y después del ajuste fino.

Se observa una mejora bastante significativa en la performance de SigLIP a pesar de haber sido ajustado con un dataset tan pequeño. Se puede concluir entonces que la cantidad de datos utilizada fue suficiente para lograr que el modelo se especialice en distinguir el dominio particular. El modelo evolucionó desde un entendimiento general de distintas comidas, para convertirse en un experto enfocado en recetas (aprendió a distinguir qué features visuales son determinantes para este caso particular).

4. Evaluación humana

Se construyó una interfaz visual con Streamlit para poder comparar las salidas de dos modelos a la vez, como se muestra en la Figura 11.

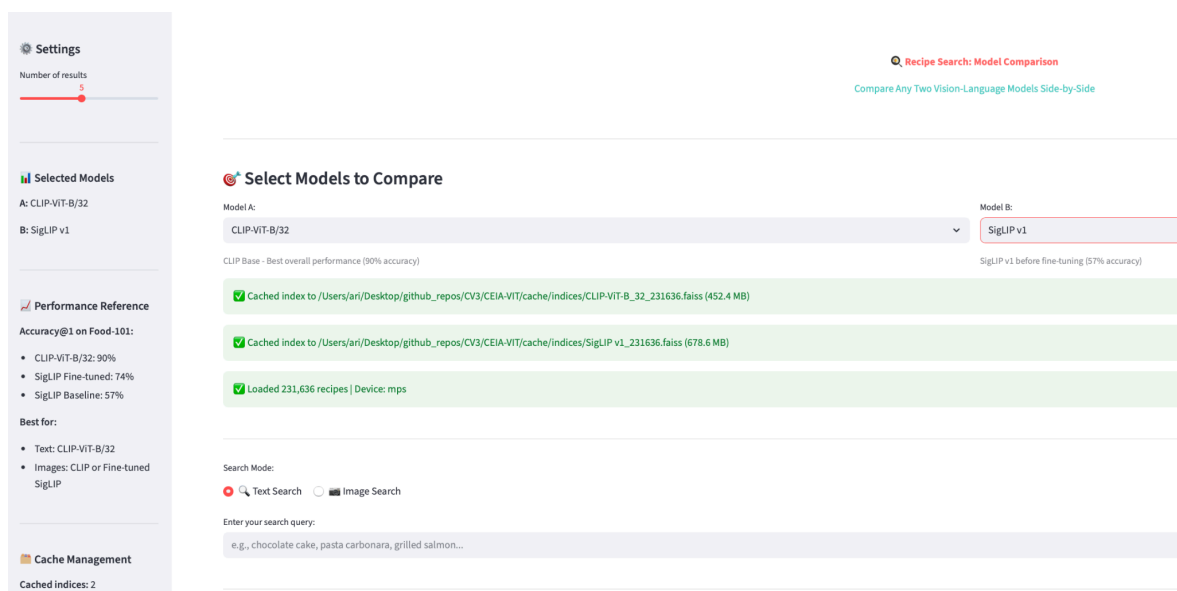


Figura 11. Aplicación para analizar resultados en forma manual y visual.

Luego, se efectuaron una serie de consultas, por medio de imágenes y prompts de texto. Para la comparación se utilizó el modelo CLIP más pequeño, y dos versiones de SigLIP (base y ajustado con fine-tuning). Además, se configuró la aplicación para entregar una única salida (la mejor, top-1).

A continuación se muestran capturas de pantalla con las salidas del sistema en cada caso.



Figura 12. Prueba con imagen de entrada: corn tortillas. (CLIP vs SigLIP v1).

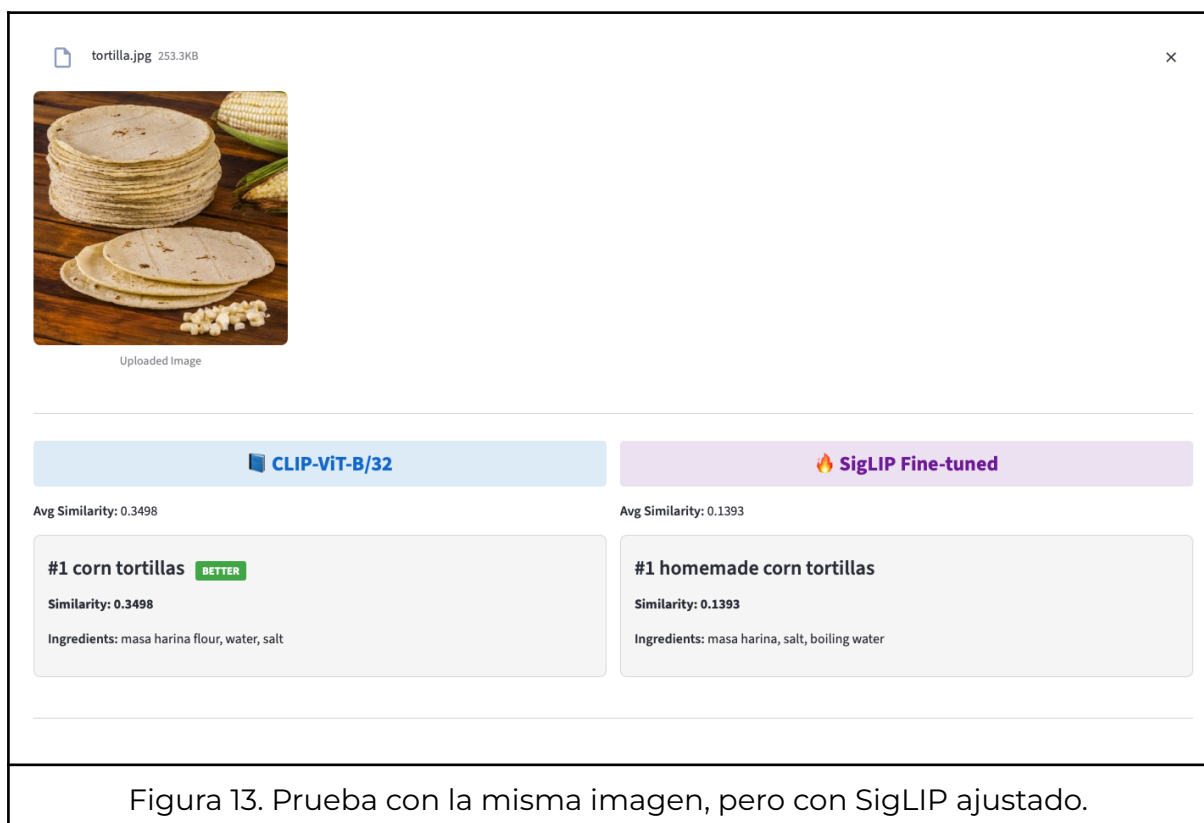


Figura 13. Prueba con la misma imagen, pero con SigLIP ajustado.

Se observa que para la misma imagen (corn tortillas), todos los modelos devuelven una receta igual de válida desde el punto de vista del usuario.



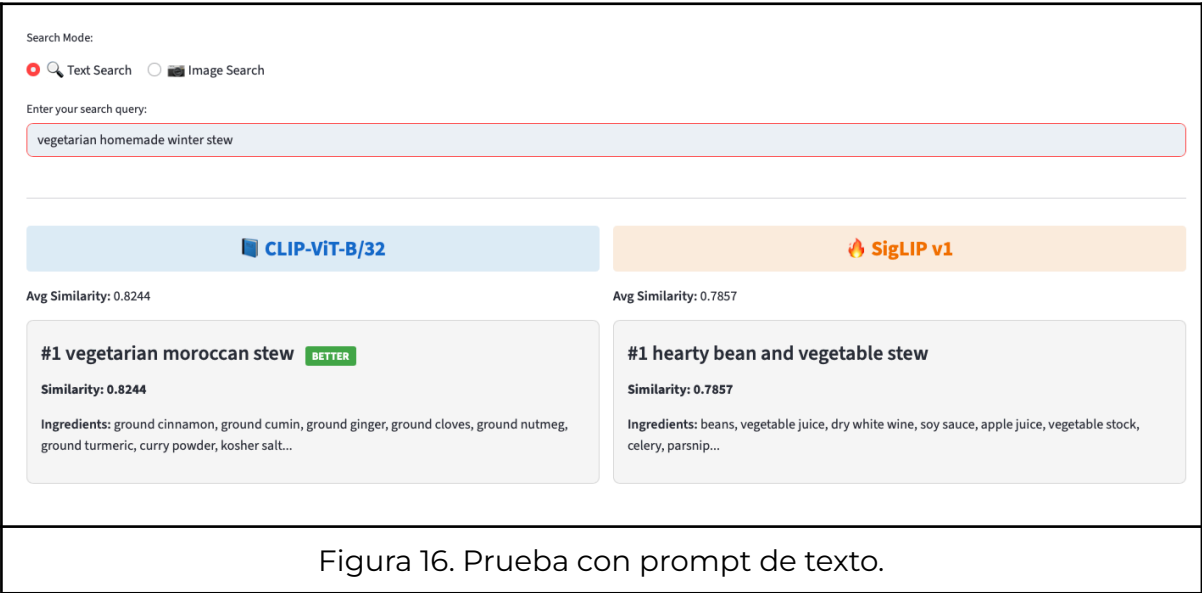
Para la imagen de la figura 14, el modelo ajustado entrega la salida más precisa desde nuestro punto de vista.



Para el caso de la figura 15, el primer modelo muestra una receta de glaseado, y el segundo, una receta de galletas con forma de donas y con glaseado. Ninguno de

los modelos devuelve la receta esperada (donas glaseadas), aunque la recomendación del modelo SigLip ajustado se acerca más a la solicitud.

Por último, se listan algunos ejemplos de queries de texto:



Ambos resultados se ajustan correctamente al pedido.



En este caso, SigLIP vuelve a recomendar una receta que se ajusta mucho mejor a lo especificado en el prompt.

Resultados

A diferencia de lo que sucede con las pruebas basadas en métricas, con las pruebas humanas consideramos que las recomendaciones del modelo SigLip, en particular el ajustado con fine-tuning, se adecua mejor a lo esperado.

Conclusiones y mejoras futuras

El proyecto demuestra de manera exitosa la viabilidad de hacer búsqueda multimodal en el dominio de recetas culinarias. Entre los modelos evaluados, CLIP emerge como una opción muy versátil, ya que ofrece un buen rendimiento general con baja latencia. SigLIP (con fine-tuning), sin embargo, fue el preferido de cara al usuario final. Por lo tanto, es importante complementar la evaluación de este tipo de sistemas con pruebas que tengan en consideración la perspectiva humana.

Algo no menor es que los recursos de hardware tienen una gran influencia a la hora de desarrollar y evaluar el sistema. Por lo tanto, planificar la infraestructura adecuada es crítico para el éxito del proyecto.

Por otra parte, es importante considerar la integración con herramientas como MLflow, que garanticen un flujo de trabajo reproducible, faciliten la experimentación iterativa y contribuyan al mantenimiento del proyecto.

En cuanto a la performance, se encontró que el sistema tiene limitaciones que se derivan principalmente del dataset subyacente (particularmente en la inconsistencia del etiquetado dietético) más que de deficiencias en la arquitectura o los modelos empleados.

A pesar de los desafíos y limitaciones encontrados, el sistema integra de forma coherente técnicas de procesamiento de lenguaje natural, visión por computadora y buenas prácticas de MLOps en una aplicación funcional, evaluable y desplegable.

Como posibles trabajos futuros, se proponen las siguientes mejoras:

- Sustituciones inteligentes mediante LLM: reemplazar el sistema basado en reglas por un modelo de lenguaje grande (Ollama o Groq) para ofrecer sustituciones contextuales más naturales y sofisticadas.
- Fine-tuning de CLIP: ajustar el modelo sobre pares imagen-receta específicos del dominio culinario para mejorar la alineación semántica.
- Sistema de feedback y aprendizaje de preferencias: incorporar valoraciones de usuarios para reentrenar o re-ranear resultados de forma personalizada.
- Exposición de los servicios mediante API REST para facilitar la integración con aplicaciones externas.
- Caché distribuida con Redis: almacenar embeddings de consultas frecuentes para reducir aún más la latencia.
- Motor de recomendación personalizada: aprender preferencias individuales del usuario a lo largo del tiempo.

- Soporte multiidioma: incorporar traducción automática de recetas y consultas mediante modelos especializados.
- Evaluación con métricas más orientadas al usuario, por ejemplo:
 - Diet Filter Compliance: porcentaje de recetas devueltas que cumplen el filtro activo (vegano, etc.).
 - Post-Filter Loss: porcentaje de candidatos relevantes (top-100 antes de filtrar) que se descartan al aplicar un filtro dietético. Mide cuánto daño hacen los filtros a la calidad del ranking.

Futura incorporación del dataset Recipe1M+

Se obtuvo acceso oficial al dataset Recipe1M+ (el mayor conjunto público de pares imagen-receta con ≈ 1 millón de ejemplos). Sin embargo, su integración no fue viable en esta versión del proyecto por dos restricciones objetivas:

1. Temporal: el acceso fue concedido cuando los plazos de entrega eran acotados. Además, al intentar la descarga, la velocidad de bajada resultó extremadamente lenta.
2. Computacional: el dataset ocupa más de 200 GB y estimamos que la generación completa de embeddings multimodales con CLIP ViT-B/32 hubiese requerido entre 40 y 80 horas-GPU en una plataforma Nvidia A100, recursos que no están disponibles.

Una posible integración de Recipe1M+ representaría la mejora de mayor impacto potencial para una versión futura del sistema.

Anexo. Planificación del equipo

Tarea	Responsable	Estado
Definición del alcance y objetivos del proyecto	Todos	Completado
Elección del dataset (Food.com 231k) y descarga	Todos	Completado
Setup inicial del entorno (Python, conda/poetry, GitHub repo, estructura de carpetas)	Martín	Completado
Setup de Databricks + MLflow (workspace, credenciales, tracking URI)	Martín	Completado
Diseño de la arquitectura general (diagrama de bloques)	Martín + revisión equipo	Completado
Decisión del stack tecnológico (CLIP vs Sentence-Transformers, FAISS, Streamlit, etc.)	Todos	Completado
Análisis exploratorio (EDA) del dataset Food.com	Carina	Completado
Enriquecimiento y corrección de tags dietéticos (inferidos desde ingredientes)	Carina	Completado
Generación del dataset final limpio (CSV + DataFrame persistido)	Carina	Completado
Experimentación y benchmarking de modelos de embeddings (MiniLM, CLIP B/32, L/14, SigLIP...)	Ariadna	Completado
Generación masiva de embeddings + guardado en .npy	Martín	Completado
Construcción y entrenamiento/tuning del índice FAISS (IVF+PQ)	Ariadna	Completado
Implementación del retriever + filtros dietéticos	Martín	Completado
Desarrollo de la interfaz Streamlit (upload imagen, texto, filtros, resultados, sustituciones)	Ariadna	Completado
Integración completa end-to-end (Streamlit, modelo, FAISS, UI)	Martín	Completado
Integración automática de logging a MLflow/Databricks (métricas, artefactos, gráficos)	Ariadna	Completado
Tests automáticos y evaluación humana (Accuracy@k, latencias, casos difíciles)	Ariadna	Completado
Documentación final, README, diagrama de arquitectura y capturas. Informe. Presentación final.	Ariadna, Carina	En curso
Sustituciones inteligentes con LLM (Ollama/Groq)	-	A futuro
Incorporación de Recipe1M+ o pares imagen-receta reales	-	A futuro
Fine-tuning de CLIP en dominio culinario	-	A futuro
Sistema de feedback/ratings de usuarios y re-ranking	-	A futuro
API REST pública (FastAPI)	-	A futuro
Caché de embeddings frecuentes con Redis	-	A futuro
Recomendaciones personalizadas (user embeddings)	-	A futuro
Soporte multiidioma y traducción automática	-	A futuro